Agentic fMRIPrep: Agents, Errors, and Recovery

Repo-grounded overview of the four-agent loop, how each stage works, the main errors encountered during toy-data and smoke runs, and how the pipeline rectified them.

Agent Loop

- Config Agent: builds the fMRIPrep Docker command from Config values, participant information, dataset facts, and local docs when RAG is available. In the current code it can proactively add flags such as --use-syn-sdc, --fs-no-reconall, and --fallback-total-readout-time when metadata inspection indicates they are needed.
- Executor: runs the generated command, captures stdout and stderr, and records a structured execution event for each attempt.
- Diagnostic Agent: reads the failure log and identifies one primary root cause with evidence and suggested fixes. If no LLM is available it falls back to heuristics for common issues such as missing readout timing, out-of-memory, fieldmap problems, FreeSurfer licensing, and Docker daemon failures.
- Recovery Agent: rewrites the failed command with targeted fixes. Examples include adding --low-mem and reducing --nprocs for memory failures, or adding --fallback-total-readout-time for missing readout timing.
- Vision / QA stage: runs only after a successful execution, resolves the real derivatives layout from the output tree, and writes qa_report.txt plus qa_metrics.json with concrete metrics instead of an arbitrary score.

How The Pipeline Works

- LangGraph state machine in agents/orchestrator.py wires the loop as planner -> executor -> detective -> engineer -> executor, with a vision step after success.
- main.py now writes publication-style provenance artifacts for each run: run_summary.json, config_snapshot.json, qa_report.txt, and qa_metrics.json under output_dir/agentic_results/sub-01/.
- Retries are bounded by max_recovery_attempts, but the graph now stops early if recovery does not produce a real command change. This prevents wasteful reruns of the same command.

Observed Errors And Rectification

- Synthetic smoke-test OOM failure: the smoke harness intentionally triggered exit code 137 with an out-of-memory message. Diagnosis labeled it OUT-OF-MEMORY crash, recovery added --low-mem and reduced --nprocs to 1, and the second attempt succeeded.
- Real toy-data metadata failure: fMRIPrep failed because BOLD metadata lacked usable readout timing. Recovery was upgraded to add --fallback-total-readout-time 0.05, and config generation was later improved to include that flag proactively before the first attempt when metadata is missing.
- Real environment failure: Docker permission or daemon access problems were correctly diagnosed as a Docker daemon / permissions error. Because recovery had no valid command-level fix, the pipeline now stops instead of looping uselessly.
- QA path failure: the original QA logic assumed outputs always lived under output_dir/fmriprep/sub-01. The code now discovers either output_dir/sub-01 or output_dir/fmriprep/sub-01, which fixed false QA misses on real toy-data runs.
- Reporting issue: the old Quality Score formula was removed. QA now reports raw nonzero voxels, preprocessed nonzero voxels, brain-mask voxels, brain-mask retention ratio, skull-strip reduction, MNI file detection, and a simple QA Summary.

Smoke Recovery Demo

- Smoke run location: tmp/full_agent_smoke/
- Attempt 1 failed with exit 137 and the message 'out of memory'.
- Diagnosis: ROOT CAUSE: OUT-OF-MEMORY crash. Evidence: out of memory.
- Recovered command added --low-mem and rewrote the command to use --nprocs 1.
- Attempt 2 succeeded and Vision / QA ran automatically afterward.

Validated Toy-Data Result

- Validated run: tmp/real_toy_outputs_pub3
- Attempts: 1. The Config Agent added --fallback-total-readout-time 0.05 proactively, so the run succeeded on the first try.
- QA Summary: PASS
- Raw voxels: 262,144
- Preprocessed voxels: 261,502
- Brain-mask voxels: 258,332
- Brain-mask retention ratio: 98.5%
- Skull-strip reduction: 1.5%
- Normalization: MNI-space preprocessed output file detected.

Why This Is Stronger Now

- The system is more autonomous because known metadata issues can be corrected before the first execution instead of only after a crash.
- The system is more honest because diagnosis focuses on the primary cause, QA is based on explicit metrics, and no-op recoveries no longer consume extra attempts.
- The system is more publishable because every run now saves provenance and QA artifacts that can be inspected, compared, and reported.

Current Limits

- This is still not a claim of perfect autonomy. Recovery is still rule-guided, not open-ended scientific reasoning.
- The smoke success path is a controlled demo using mock Docker. It proves the orchestration loop, not full scientific validity.
- A strong journal paper would still need multi-dataset evaluation, baselines, quantitative recovery rates, and external QA validation beyond the current heuristic checks.

Useful Output Files

- Smoke log: tmp/full_agent_smoke/smoke_run.log
- Toy-data run summary: tmp/real_toy_outputs_pub3/agentic_results/sub-01/run_summary.json
- Toy-data config snapshot: tmp/real_toy_outputs_pub3/agentic_results/sub-01/config_snapshot.json
- Toy-data QA report: tmp/real_toy_outputs_pub3/agentic_results/sub-01/qa_report.txt
- Toy-data QA metrics: tmp/real_toy_outputs_pub3/agentic_results/sub-01/qa_metrics.json